

2025ACM寒假第一次讲课

时空复杂度

用途：

分析程序的效率

定义：

时空复杂度又分为时间复杂度和空间复杂度

时间复杂度的全称是渐进时间复杂度,表示算法的**执行时间与数据规模之间的增长关系**。

//执行次数

//称n为问题的规模,当n不断变化时,它所呈现出来的执行次数规律,我们称之为时间复杂度

空间复杂度全称就是渐进空间复杂度,表示算法的**存储空间与数据规模之间的增长关系**。

//对一个算法在运行过程中临时占用存储空间大小的量度

//这里的存储空间是指程序运行过程的所有变量申请的空间

//指的是程序在运行时所需要的 **最大空间占用**

- 栈空间：自动管理内存,用于存储函数的局部变量、递归函数调用等。
- 堆空间：动态分配,由程序员通过 `new` 或 `malloc` 手动管理。
- 全局变量：存储全局的常量和静态变量。

//称n为问题的规模,当n不断变化时,它所呈现出来的存储空间大小规律,我们称之为空间复杂度

表示：

表示复杂度——大O复杂度表示法

时间复杂度表示法：

一个算法中的执行次数称为语句频度或时间频度,记为 $T(n)$

那么大O复杂度表示为：

$$T(n) = O(f(n))$$

n表示数据规模的大小;

$f(n)$ 表示每行代码执行的次数总和;

O代表代码的执行时间

$T(n)$ 与 $f(n)$ 表达式成正比。

例如：

```
cin>>n;
for(int i = 0; i < n; ++i) {
```

```

        //n
    }
    for(int k = 0; k < 10; ++k)//10
    for(int i = 0; i < n; ++i) { //n
        for(int j = 0; j < n; ++j) { //n
            //10*n*n
        }
    }
}

```

$$f(n) = 10n^2 + n, O(f(n)) = O(10n^2 + n) = O(n^2)$$

可以说这个程序是 $O(n^2)$ 的时间复杂度

时间复杂度只关注最高数量级，且与之系数也没有关系

也就是说当我们分析时间复杂度时

1. 只需关注执行次数最多的一段代码；
2. 嵌套代码的复杂度等于嵌套内外代码复杂度的乘积。

同理空间复杂度也可以用大 O 复杂度表示法

栈空间大小通常在 **2 MB 到 8 MB** 范围内，具体依赖于评测系统设置。

注意栈空间的分析，它的最大搜索深度 * 使用的变量空间

```

    for (int i = 0; i < n; ++i) {

        int a[n];

    }

    int f(int n) {
        if(n == 0) return 1;
        if(n == 1) return 1;
        return n*f(n-1);
    }

```

分析

最好情况时间复杂度

最坏情况时间复杂度

平均时间复杂度

1. 常见复杂度排序（由小到大）

复杂度	名称	示例
$O(1)$	常数级复杂度	查找哈希表、访问数组元素
$O(\log n)$	对数级复杂度	二分查找、平衡二叉树的插入
$O(n)$	线性复杂度	遍历数组、简单搜索
$O(n \log n)$	线性对数复杂度	高效排序（如归并排序、快速排序）
$O(n^2)$	二次复杂度	双重嵌套循环、朴素矩阵乘法
$O(2^n)$	指数级复杂度	递归求解斐波那契（未优化）
$O(n!)$	阶乘级复杂度	全排列生成，旅行商问题的暴力求解

2. 不同复杂度的增长对比

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(2^n)$	$O(n!)$
10	1	4	10	40	100	1024	3,628,800
100	1	7	100	700	10,000	1.27×10^{30}	超大
1000	1	10	1000	10,000	1,000,000	超大	超大

实际运用

由数据范围反推算法复杂度以及算法内容

作者:  yxc, 2018-09-10 22:27:18, 所有人可见, 阅读 80338



一般ACM或者笔试题的时间限制是1秒或2秒。
在这种情况下，C++代码中的操作次数控制在 $10^7 \sim 10^8$ 为最佳。

3827



下面给出在不同数据范围下，代码的时间复杂度和算法该如何选择：



6379

- $n \leq 30$, 指数级别, dfs+剪枝, 状态压缩dp
- $n \leq 100 \Rightarrow O(n^3)$, floyd, dp, 高斯消元
- $n \leq 1000 \Rightarrow O(n^2)$, $O(n^2 \log n)$, dp, 二分, 朴素版Dijkstra, 朴素版Prim, Bellman-Ford
- $n \leq 10000 \Rightarrow O(n * \sqrt{n})$, 块状链表、分块、莫队
- $n \leq 100000 \Rightarrow O(n \log n) \Rightarrow$ 各种sort, 线段树、树状数组、set/map、heap、拓扑排序、dijkstra+heap、prim+heap、Kruskal、spfa、求凸包、求半平面交、二分、CDQ分治、整体二分、后缀数组、树链剖分、动态树
- $n \leq 1000000 \Rightarrow O(n)$, 以及常数较小的 $O(n \log n)$ 算法 $\Rightarrow$ 单调队列、hash、双指针扫描、BFS、并查集、kmp、AC自动机, 常数比较小的 $O(n \log n)$ 的做法: sort、树状数组、heap、dijkstra、spfa
- $n \leq 10000000 \Rightarrow O(n)$, 双指针扫描、kmp、AC自动机、线性筛素数
- $n \leq 10^9 \Rightarrow O(\sqrt{n})$, 判断质数
- $n \leq 10^{18} \Rightarrow O(\log n)$, 最大公约数, 快速幂, 数位DP
- $n \leq 10^{1000} \Rightarrow O((\log n)^2)$, 高精度加减乘除
- $n \leq 10^{100000} \Rightarrow O(\log k \times \log \log k)$, k 表示位数, 高精度加减、FFT/NTT

空间上：

通常限制是**64 MB - 512 MB**。

bit（位，又名“比特”）：**bit**的缩写是**b**，是计算机中的最小数据单位（属于二进制的范畴，其实就是**0**或者**1**）

Byte（字节）：**Byte**的缩写是**B**，是计算机文件大小的基本计算单位。比如一个字符就是**1Byte**，如果是汉字，则是**2Byte**。

1TB = (1*1024) GB = (1*1024*1024) MB = (1*1024*1024*1024) KB =

```
(1*1024*1024*1024*10224) B
```

```
64MB = 67108864B = 16777216(int) = 4096*4096(int)
```

一般不会卡你空间

模拟

模拟就是用计算机来模拟题目中要求的操作。

模拟题目通常具有码量大、操作多、思路繁复的特点。由于它码量大，经常会出现难以查错的情况，如果在考试中写错是相当浪费时间的。

- 在动手写代码之前，在草纸上尽可能地写好要实现的流程。
- 写代码的时候一定要思路清晰，不要想到什么写什么，要按照落在纸上的步骤写。
- 实现代码结构化，分层化，模板化，写成函数，类，结构体

洛谷P1328: <https://www.luogu.com.cn/problem/P1328>

洛谷P1067: <https://www.luogu.com.cn/problem/P1067>

枚举

枚举（英语：Enumerate）是基于已有知识来猜测答案的一种问题求解策略。

枚举的思想是不断地猜测，从可能的集合中一一尝试，然后再判断题目的条件是否成立

枚举算法的核心思想是：通过列举问题的所有状态，将它们逐一与目标状态进行比较，从而得到满足条件的解。

// 遍历某些值，判断合理性

[P1145 约瑟夫 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P1145>)

[P1387 最大正方形 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P1387>)

[P1149 [NOIP2008 提高组] 火柴棒等式 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P1149>)

[P3799 小 Y 拼木棒 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P3799>)

递归

定义

递归（英语：Recursion），在数学和计算机科学中是指在函数的定义中使用函数自身的方法，在计算机科学中还额外指一种通过重复将问题分解为同类的子问题而解决问题的方法。

递归的基本思想是**某个函数直接或者间接地调用自身**，这样原问题的求解就转换为了许多性质相同但是规模更小的子问题。求解时只需要关注如何把原问题划分成符合条件的子问题，而不需要过分关注这个子问题是如何被解决的。

明白一个函数的作用并相信它能完成这个任务，千万不要跳进这个函数里面企图探究更多细节
递归代码最重要的两个特征：结束条件和自我调用。自我调用是在解决子问题，而结束条件定义了最小子问题的答案。

```
int func(传入数值) {  
    if (终止条件)  
        return 最小子问题解;  
    return func(缩小规模);  
}
```

例子

经典的递归就求阶乘

```
int f(int n) { //n!  
    if(n == 0) return 1;  
    if(n == 1) return 1;  
    return n*f(n-1); //f(n)直接调用f(n-1), f(n)递归到f(n-1)  
}
```

递归的缺点

在程序执行中，递归是利用堆栈来实现的。每当进入一个函数调用，栈就会增加一层栈帧，每次函数返回，栈就会减少一层栈帧。而栈不是无限大的，当递归层数过多时，就会造成**栈溢出**的后果。

递归的优化

剪枝 + 记忆化搜索

```
[P1255 数楼梯 - 洛谷 | 计算机科学教育新生态]  
(https://www.luogu.com.cn/problem/P1255)  
[P1044 [NOIP2003 普及组] 栈 - 洛谷 | 计算机科学教育新生态]  
(https://www.luogu.com.cn/problem/P1044)  
[P1028 [NOIP2001 普及组] 数的计算 - 洛谷 | 计算机科学教育新生态]  
(https://www.luogu.com.cn/problem/P1028)  
[P1464 Function - 洛谷 | 计算机科学教育新生态]  
(https://www.luogu.com.cn/problem/P1464)
```

[P1010 [NOIP1998 普及组] 幂次方 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P1010>)

[P1928 外星密码 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P1928>)

[P1990 覆盖墙壁 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P1990>)

分治

定义

分治（英语：Divide and Conquer），字面上的解释是「分而治之」，就是把一个复杂的问题分成两个或更多的相同或相似的子问题，直到最后子问题可以简单的直接求解，原问题的解即子问题的解的合并。

过程

分治算法的核心思想就是「分而治之」。

大概的流程可以分为三步：**分解** -> **解决** -> **合并**。

1. 分解原问题为结构相同的子问题。
2. 分解到某个容易求解的边界之后，进行递归求解。
3. 将子问题的解合并成原问题的解。

分治法能解决的问题一般有如下特征：

- 该问题的规模缩小到一定的程度就可以容易地解决。
- 该问题可以分解为若干个规模较小的相同问题，即该问题具有最优子结构性质，利用该问题分解出的子问题的解可以合并为该问题的解。
- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

递归与分治的区别

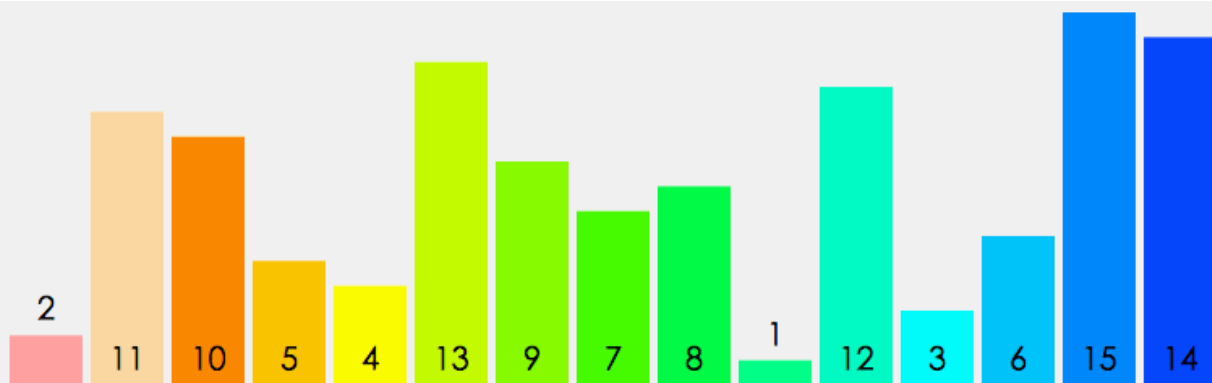
递归是一种编程技巧，一种解决问题的思维方式；分治算法很大程度上是基于递归的，解决更具体问题的算法思想

例子

归并排序基于分治思想将数组分段排序后合并

也就是给一个数组排序等于给该数组的左右两半分别排序，然后合并成一个数组。

时间复杂度在最优、最坏与平均情况下均为 $O(N\log N)$ ，空间复杂度为 $O(N)$



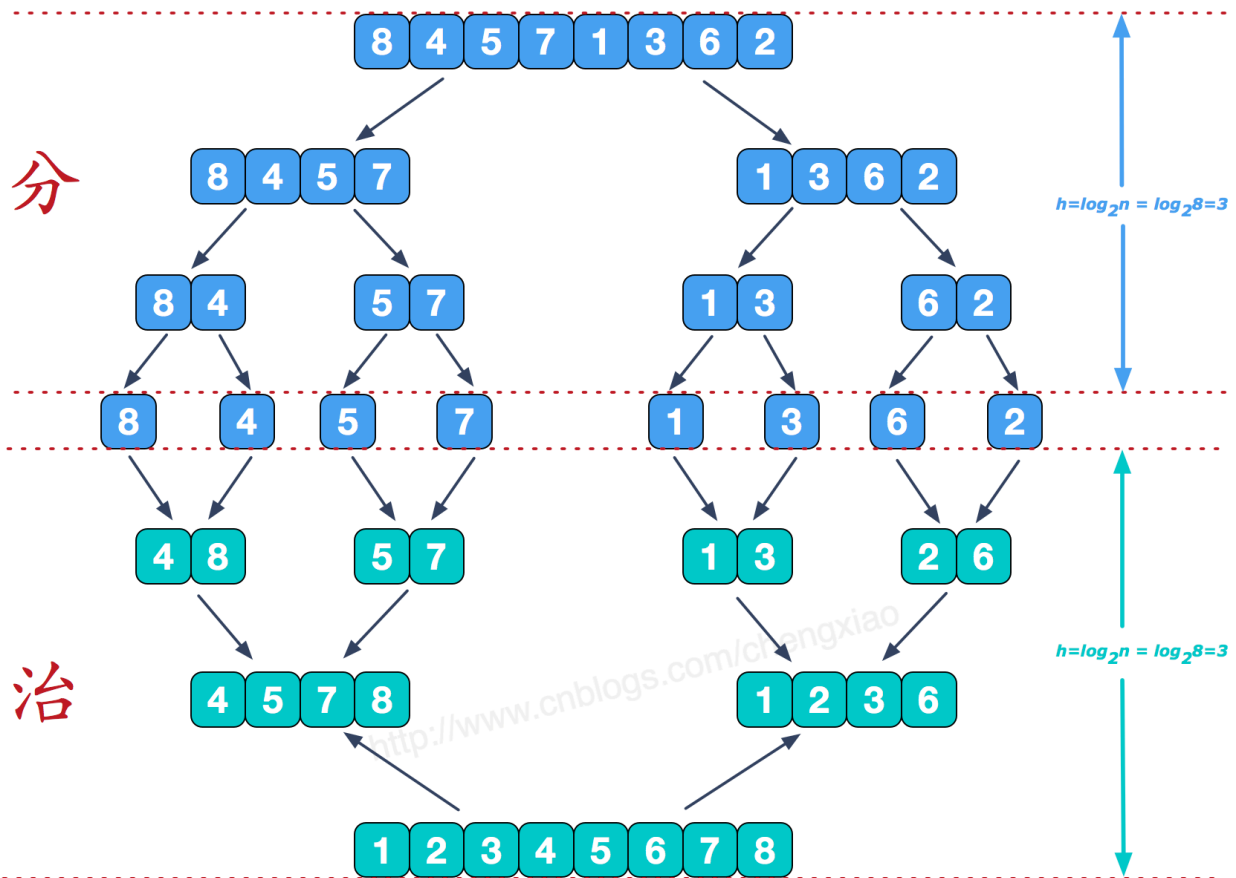
```
void merge_sort(int q[], int l, int r)
{
    if (l >= r) return;

    int mid = l + r >> 1;
    merge_sort(q, l, mid);
    merge_sort(q, mid + 1, r);

    int k = 0, i = l, j = mid + 1;
    while (i <= mid && j <= r)
        if (q[i] < q[j]) tmp[k ++ ] = q[i ++ ];
        else tmp[k ++ ] = q[j ++ ];

    while (i <= mid) tmp[k ++ ] = q[i ++ ];
    while (j <= r) tmp[k ++ ] = q[j ++ ];

    for (i = l, j = 0; i <= r; i ++, j ++ ) q[i] = tmp[j];
}
```



<https://blog.csdn.net/jusidie>

双指针

双指针顾名思义，就是同时使用两个指针，在序列、链表结构上指向的是位置，在树、图结构中指向的是节点，通过或同向移动，或相向移动来维护、统计信息。

例子

最经典的例子是快速排序

快速排序是一种不稳定的排序算法。

快速排序的最优时间复杂度和平均时间复杂度为 $O(N \log N)$ ，最坏时间复杂度为 $O(N^2)$ 。

快速排序的工作原理是通过分治+双指针的方式来将一个数组排序。

快速排序分为三个过程：

1. 将数列划分为两部分（要求保证相对大小关系）；
2. 递归到两个子序列中分别进行快速排序；

3. 不用合并，因为此时数列已经完全有序。

@五分钟学算法之快速排序

```
void quick_sort(int q[], int l, int r)
{
    if (l >= r) return;

    int i = l - 1, j = r + 1, x = q[l + r >> 1];
    while (i < j)
    {
        do i ++ ; while (q[i] < x);
        do j -- ; while (q[j] > x);
        if (i < j) swap(q[i], q[j]);
    }
    quick_sort(q, l, j), quick_sort(q, j + 1, r);
}
```

[P1177 【模板】排序 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P1177>)

[Problem - 1385D - Codeforces]

(<https://codeforces.com/problemset/problem/1385/D>)

[P1885 Moo - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P1885>)

[Problem - 1955D - Codeforces]

(<https://codeforces.com/problemset/problem/1955/D>)

倍增

定义

倍增是种成倍增长的思想

最常用的倍增思想是二进制划分

也就是2这个数倍增来划分一个整数

$$//10 = 2^3 + 2^1$$

例子

快速幂

快速幂，二进制取幂（Binary Exponentiation，也称平方），是一个在 $O(\log N)$ 的时间内计算的小技巧，而暴力的计算需要 $O(N)$ 的时间。

例如求 100^{10}

暴力：10个100相乘

快速幂：以2倍增，划分整数10

$$10 = 2^3 + 2^1$$

进而

$$100^{10} = 100^{(1010)_2} = 100^8 * 100^2$$

枚举100的0, 2, 4, 8次幂，选择10的二进制数位为1所对应的次幂相乘即可共4次

模板代码

```
int ksm(int a,int b) { //a^b
    int res = 1;
    while(b) {
        if(b&1) {
            res *= a;
        }
        a = a*a; //倍增a - a^2 - a^4 ...
        b >>= 1;
    }
    return res;
}
```

[[ABC367E] Permute K times - 洛谷 | 计算机科学教育新生态]

(https://www.luogu.com.cn/problem/AT_abc367_e)

[Problem - 1516D - Codeforces]

(<https://codeforces.com/problemset/problem/1516/D>)

链表

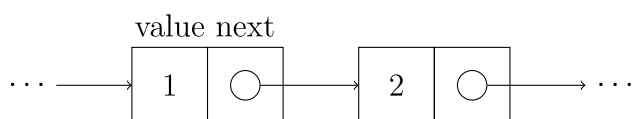
链表是一种用于存储数据的数据结构，通过如链条一般的**指针来连接元素**。它的特点是插入与删除数据十分方便，但寻找与读取数据的表现欠佳。

链表因其链状的结构，能方便地删除、插入数据，操作次数是 $O(1)$ 。但也因为这样，寻找、读取数据的效率不如数组高，在随机访问数据中的操作次数是 $O(N)$ 。

数组可以方便地寻找并读取数据，在随机访问中操作次数是 $O(1)$ 。但删除、插入的操作次数是 $O(N)$ 次。

单向链表

单向链表中包含数据域和指针域，其中数据域用于存放数据，指针域用来连接当前结点和下一节点。



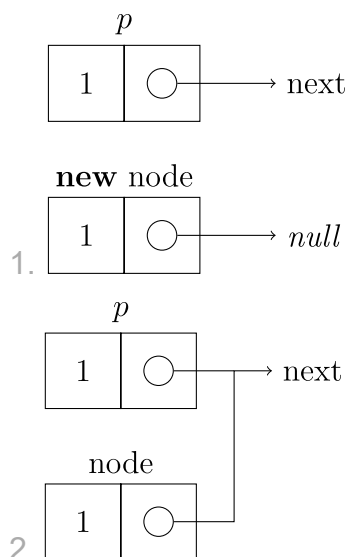
```
struct Node {  
    int value;  
    Node *next;  
};
```

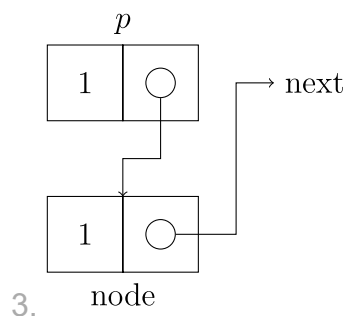
插入元素

流程大致如下：

1. 初始化待插入的数据 `node`；
2. 将 `node` 的 `next` 指针指向 `p` 的下一个结点；
3. 将 `p` 的 `next` 指针指向 `node`。

具体过程可参考下图：





代码实现如下：

```
void insertNode(int i, Node *p) { //在p后面添加一个元素
    Node *node = new Node;
    node->value = i;
    node->next = p->next;
    p->next = node;
}
```

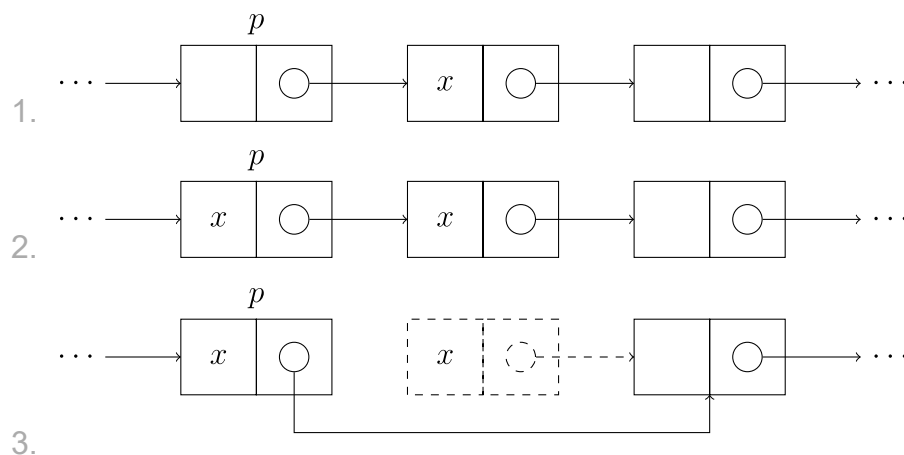
删除元素

设待删除结点为 p ，从链表中删除它时，将 p 的下一个结点 $p \rightarrow next$ 的值覆盖给 p 即可，与此同时更新 p 的下下个结点。

流程大致如下：

1. 将 p 下一个结点的值赋给 p ，以抹掉 $p \rightarrow value$ ；
2. 新建一个临时结点 t 存放 $p \rightarrow next$ 的地址；
3. 将 p 的 $next$ 指针指向 p 的下下个结点，以抹掉 $p \rightarrow next$ ；
4. 删除 t 。此时虽然原结点 p 的地址还在使用，删除的是原结点 $p \rightarrow next$ 的地址，但 p 的数据被 $p \rightarrow next$ 覆盖， p 名存实亡。

具体过程可参考下图：

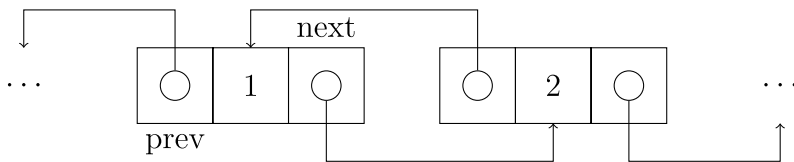


代码实现如下：

```
void deleteNode(Node *p) { //删除p
    p->value = p->next->value;
    Node *t = p->next;
    p->next = p->next->next;
    delete t;
}
```

双向链表

双向链表中同样有数据域和指针域。不同之处在于，指针域有左右（或上一个、下一个）之分，用来连接上一个结点、当前结点、下一个结点。



```
struct Node {
    int value;
    Node *left;
    Node *right;
};
```

插入元素

大致流程如下：

1. 初始化待插入的数据 `node` ；
2. 将 `node` 的 `left` 指针指向 `p` ；
3. 将 `node` 的 `right` 指针指向 `p` 的右结点；
4. 将 `p` 右结点的 `left` 指针指向 `node` ；
5. 将 `p` 的 `right` 指针指向 `node` 。

代码实现如下：

```
void insertNode(int i, Node *p) { //在p后插入
    Node *node = new Node;
    node->value = i;
    node->left = p;
    node->right = p->right;
    p->right->left = node;
    p->right = node;
}
```

删除元素

流程大致如下：

1. 将 `p` 左结点的右指针指向 `p` 的右节点；
2. 将 `p` 右结点的左指针指向 `p` 的左节点；
3. 新建一个临时结点 `t` 存放 `p` 的地址；
4. 将 `p` 的右节点地址赋给 `p`，以避免 `p` 变成^[1]悬垂指针；
5. 删除 `t`。

```
void deleteNode(Node *&p) {  
    //传入需删除结点引用，删除p后，将p指向它下一个元素，避免 p 变成悬垂指针  
    p->left->right = p->right;  
    p->right->left = p->left;  
    Node *t = p;  
    p = p->right;  
    delete t;  
}
```

单向循环链表

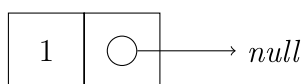
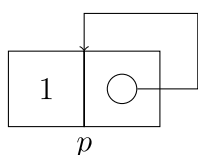
将链表的头尾连接起来，链表就变成了循环链表。

由于链表首尾相连，在插入数据时需要判断原链表是否为空：为空则自身循环，不为空则正常插入数据。

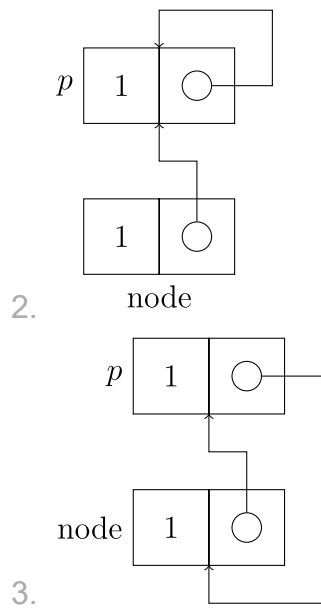
大致流程如下：

1. 初始化待插入的数据 `node`；
2. 判断给定链表 `p` 是否为空；
3. 若为空，则将 `node` 的 `next` 指针和 `p` 都指向自己；
4. 否则，将 `node` 的 `next` 指针指向 `p` 的下一个结点；
5. 将 `p` 的 `next` 指针指向 `node`。

具体过程可参考下图：



1. `new node`



代码实现如下：

```
void insertNode(int i, Node *p) {
    Node *node = new Node;
    node->value = i;
    node->next = NULL;
    if (p == NULL) {
        p = node;
        node->next = node;
    }
    else {
        node->next = p->next;
        p->next = node;
    }
}
```

[B3631 单向链表 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/B3631>)

[P10466 邻值查找 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P10466>)

手动建立

单链表

// head存储链表头，e[]存储节点的值，ne[]存储节点的next指针，idx表示当前用到了哪个节点

```
int head, e[N], ne[N], idx;
```

// 初始化

```
void init()
```

```
{
```

```
    head = -1;
```

```

    idx = 0;
}

// 在链表头插入一个数a
void insert(int a)
{
    e[idx] = a, ne[idx] = head, head = idx ++ ;
}

// 将头结点删除，需要保证头结点存在
void remove()
{
    head = ne[head];
}

```

双链表

// e[]表示节点的值，l[]表示节点的左指针，r[]表示节点的右指针，idx表示当前用到了哪个节点

```
int e[N], l[N], r[N], idx;
```

// 初始化

```
void init()
{
    //0是左端点，1是右端点
    r[0] = 1, l[1] = 0;
    idx = 2;
}

```

// 在节点a的右边插入一个数x

```
void insert(int a, int x)
{
    e[idx] = x;
    l[idx] = a, r[idx] = r[a];
    l[r[a]] = idx, r[a] = idx ++ ;
}

```

// 删除节点a

```
void remove(int a)
{
    l[r[a]] = l[a];
    r[l[a]] = r[a];
}

```

STL

C++ 标准模板库 (Standard Template Library, STL) 是一套功能强大的 C++ 模板类和函数的集合, 它提供了一系列通用的、可复用的算法和数据结构。

- STL提供了六大组件, 彼此之间可以组合套用, 这六大组件分别是:容器、算法、迭代器、仿函数、适配器 (配接器)、空间配置器。

我们只了解其中的容器、算法、迭代器

容器: 各种数据结构, 如vector、list、deque、set、map等,用来存放数据, 从实现角度来看, STL容器是一种class template。

算法: 各种常用的算法, 如sort、find、copy。从实现的角度来看, STL算法是一function template。

迭代器: 扮演了容器与算法之间的胶合剂, 共有五种类型, 从实现角度来看, 迭代器是一种将operator, operator->, operator++, operator--等指针相关操作予以重载的class template。
可以将迭代器看成一种指针, 可以用it表示指向元素

vector

vector, 变长数组, 倍增的思想//每当空间不够时将当前空间乘以二

vector a(n); //(n,2) 长n数为二

size() 返回元素个数/

empty() 返回是否为空/o(1)所有stl都有

clear() 清空//vector v(0);

front()/back()

push_back()/pop_back()

begin()/end()

[]

支持比较运算, 按字典序 < > ==

sort(vec.begin(), vec.end());

自定义

```
vector<int> vec = {5, 3, 8, 1, 7};
```

```
sort(vec.begin(), vec.end(), [](int a, int b) { return a > b; }); // 降序排序
```

lower_bound()/upper_bound()

lower_bound(b,e,x) 返回大于等于x的最小的数的迭代器

upper_bound(x) 返回大于x的最小的数的迭代器

lower_bound(v.begin(), v.end(), 1)) - v.begin()

去重:

```
sort(vec.begin(), vec.end());
```

```
vec.erase(unique(vec.begin(), vec.end()), vec.end());
```

```
for(vector::iterator i = a.begin(); i != a.end(); ++i) cout<<*i;
```

```
for(auto x : a)cout<<x<<endl;
```

erase

it=vec.erase(it); //删除元素, 返回值指向已删除元素的下一个位置

pair<int,int>

```
pair<int, int>
//二元组
first, 第一个元素
second, 第二个元素
支持比较运算, 以first为第一关键字, 以second为第二关键字 (字典序) < > ==
pair<int,int> p;
p = make_pair(10,2);
P = {10,2}; //C11
```

string

```
string, 字符串
string是动态大小的, 可以自动调整大小以容纳不同长度的字符串。
reverse(s.begin(),s.end());//将字符串开头和结尾反转.
substr(起始下标, (子串长度)) 返回子串 a.substr(0,2);
s.erase(pos1,len); //删除字符串s中从位置pos1,长为len的字符串->s
s.insert(1, "12"); // 在1下标添加字符串, 原一变为str后->s
string c(5, ' '); // 初始化为长度为5的空格字符串
s.find("121"); //find字符或字符串,std::string::npos (std::string::npos 是 std::size_t 类型的常量, 表示一个很大的无符号整数值 (通常为 -1 的无符号表示) 。) 或起始下标
size()/length() 返回字符串长度
string s;
getline(cin, s);//读一行
empty()
find()
clear()
c_str() 返回字符串所在字符数组的起始地址 printf("%s",a.c_str());
string a = "12";
a += "2";
int intStr = atoi(str.c_str());
atol(long long )
string str = to_string(number);
```

queue

```
queue, 队列,先进先出
size()
empty()
front() 返回队头元素//非top();
push() 向队尾插入一个元素
back() 返回队尾元素
```

pop() 弹出队头元素
无clear() q = queue();

priority_queue

priority_queue, 优先队列, 默认是大根堆

堆是一种树状数据结构,是一棵完全二叉树, 所有层都被填满, 除了最后一层, 且最后一层节点从左到右排列紧凑。

大根堆: 每个节点的值都大于或等于其子节点的值。

push() 插入一个元素

top() 返回堆顶元素

pop() 弹出堆顶元素

定义成小根堆的方式: `priority_queue<int, vector<int>, greater<int>> q;`

自定义堆:

```
auto cmp = [](int a, int b) { return a > b; }; // 小顶堆 (降序)
priority_queue<int, vector<int>, decltype(cmp)> pq(cmp);
```

stack

stack, 栈, 后进先出

size()

empty()

push() 向栈顶插入一个元素

top() 返回栈顶元素

pop() 弹出栈顶元素 //无clear();

deque

deque, 双端队列

size()

empty()

clear()

front()/back()

push_back()/pop_back()

push_front()/pop_front()

begin()/end()

[]

sort(q.begin(), q.end())//从小到大

sort(q.begin(), q.end(), greater());//从大到小排序

栈

// tt表示栈顶

```
int stk[N], tt = 0;
```

```
// 向栈顶插入一个数
```

```
stk[ ++ tt] = x;
```

```
// 从栈顶弹出一个数
```

```
tt -- ;
```

```
// 栈顶的值
```

```
stk[tt];
```

```
// 判断栈是否为空
```

```
if (tt > 0)
```

```
{
```

```
}
```

队列

1. 普通队列:

```
// hh 表示队头, tt表示队尾
```

```
int q[N], hh = 0, tt = -1;
```

```
// 向队尾插入一个数
```

```
q[ ++ tt] = x;
```

```
// 从队头弹出一个数
```

```
hh ++ ;
```

```
// 队头的值
```

```
q[hh];
```

```
// 判断队列是否为空
```

```
if (hh <= tt)
```

```
{
```

```
}
```

2. 循环队列

```
// hh 表示队头, tt表示队尾的后一个位置
```

```
int q[N], hh = 0, tt = 0;
```

```
// 向队尾插入一个数
```

```
q[tt ++ ] = x;
```

```
if (tt == N) tt = 0;
```

```
// 从队头弹出一个数
```

```
hh ++ ;
```

```
if (hh == N) hh = 0;
```

```
// 队头的值
```

```
q[hh];
```

```
// 判断队列是否为空
if (hh != tt)
{

}
```

set/map

set, map, multiset, multimap, 基于平衡二叉树（红黑树），动态维护有序序列

- set: 集合，不允许重复元素。
- multiset: 多重集合，允许多个元素具有相同的键。
- map: 映射，每个键映射到一个值。
- multimap: 多重映射，存储了键值对（pair），其中键是唯一的，但值可以重复，允许一个键映射到多个值。

size()

empty()

clear()

begin()/end()

++, -- 返回前驱和后继，时间复杂度 $O(\log n)$

set/multiset

insert() 插入一个数

find() 查找一个数

count() 返回某一个数的个数

erase()

(1) 输入是一个数x，删除所有x $O(k + \log n)$

(2) 输入一个迭代器，删除这个迭代器

lower_bound()/upper_bound()

lower_bound(x) 返回大于等于x的最小的数的迭代器

upper_bound(x) 返回大于x的最小的数的迭代器

自定义

```
auto cmp = [](int a, int b) { return a > b; }; // 降序排序
set<int, decltype(cmp)> Set(cmp);
```

map/multimap

insert() 插入的数是一个pair

erase() 输入的参数是pair或者迭代器

erase(key)

find(key)->f/s

count(key); //返回key的元素数/1,0;

[] 注意multimap不支持此操作。

时间复杂度是 $O(\log n)$

`lower_bound()/upper_bound()`

自定义

```
auto cmp = [](int a, int b) { return a > b; }; // 键降序
map<int, string, decltype(cmp)> Map(cmp);
```

`unordered_set`, `unordered_map`, `unordered_multiset`, `unordered_multimap`, 哈希表
和上面类似, 增删改查的时间复杂度是 $O(1)$

不支持 `lower_bound()/upper_bound()`, 迭代器的++, --

bitset

bitset压位 将字节压于位/8 ,

`bitset<10000> s; //<>: 个数 ; bitset 整数`

`~, &, |, ^`

`>>, <<`

`==, !=`

`[]`

`count()` 返回有多少个1

`any()` 判断是否至少有一个1

`none()` 判断是否全为0

`set()` 把所有位置成1

`set(k, v)` 将第k位变成v

`reset()` 把所有位变成0

`flip()` 等价于~

`flip(k)` 把第k位取反

[P1165 日志分析 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P1165>)

[P1540 [NOIP2010 提高组] 机器翻译 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P1540>)

[P1638 逛画展 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P1638>)

[P1714 切蛋糕 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P1714>)

[P2085 最小函数值 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P2085>)

[P1059 [NOIP2006 普及组] 明明的随机数 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P1059>)

[P1978 集合 - 洛谷 | 计算机科学教育新生态]

(<https://www.luogu.com.cn/problem/P1978>)

参考

[计算机中 b、KB、MB、GB、TB之间的换算关系-腾讯云开发者社区-腾讯云]

(<https://cloud.tencent.com/developer/article/1870970>)

[由数据范围反推算法复杂度以及算法内容 - AcWing]

(https://www.acwing.com/file_system/file/content/whole/index/content/3074/)

[复杂度分析——时间复杂度和空间复杂度（超详细总结）_空间复杂度分析-CSDN博客]

(<https://blog.csdn.net/Charlesssyang/article/details/110362769>)

[一文看懂算法的时间和空间复杂度分析 - 知乎]

(<https://zhuanlan.zhihu.com/p/143358017>)

chat

oi--wiki

[浅说倍增算法——线性倍增-CSDN博客]

(<https://blog.csdn.net/CylMK/article/details/143118784>)

[排序——归并排序（Merge sort）-CSDN博客]

(<https://blog.csdn.net/justidle/article/details/104203958>)

[快速排序算法动图演示及解析2021版（附Java代码实现）_快速排序动图-CSDN博客]

(https://blog.csdn.net/qq_38082146/article/details/115453732)

acwing

-
1. 悬垂指针是 C++ 中一个危险的现象,指的是指向已经被释放(或销毁) 内存的指针。当程序试图访问这块已释放的内存时, 会导致未定义行为 (Undefined Behavior) 。 ↩